

 TutorialIntroduction to *polars-bio* features

Interval operations illustrated

Please note that for visualizaiton of the intervals we used `plot_intervals` method from `Bioframe` package.

```
In [1]: import polars_bio as pb
import pandas as pd
import polars as pl
import bioframe as bf
```

```
In [2]: cols = ["contig", "pos_start", "pos_end"]
```

```
In [6]: df1 = pd.DataFrame(
    [ ["chr1", 1, 5], ["chr1", 3, 8], ["chr1", 8, 10],
    ["chr1", 12, 14]],
    columns=["chrom", "start", "end"],
)

df2 = pd.DataFrame(
    [ ["chr1", 4, 8], ["chr1", 10, 11]], columns=
    ["chrom", "start", "end"]
)
```

```
In [7]: display(df1)
```

	chrom	start	end
0	chr1	1	5
1	chr1	3	8
2	chr1	8	10
3	chr1	12	14

In [8]: `display(df2)`

	chrom	start	end
0	chr1	4	8
1	chr1	10	11

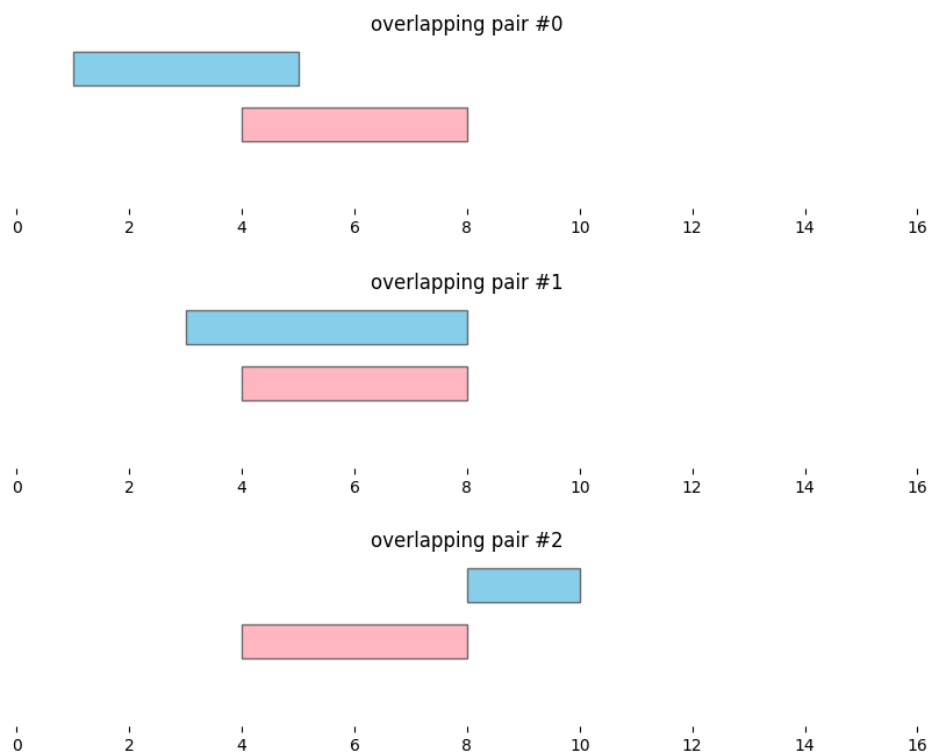
Overlap

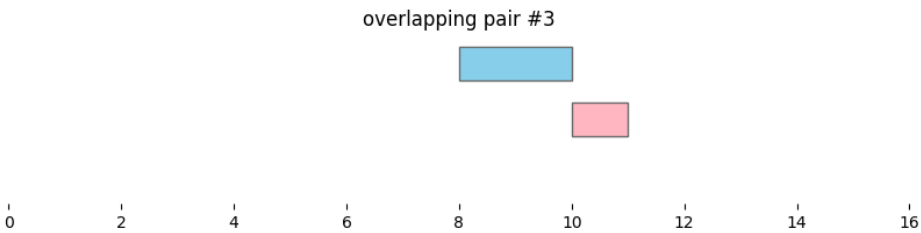
In [9]: `overlapping_intervals = pb.overlap(df1, df2, output_type="pandas.DataFrame")`

In [10]: `display(overlapping_intervals)`

	chrom_1	start_1	end_1	chrom_2	start_2	end_2
0	chr1	1	5	chr1	4	8
1	chr1	3	8	chr1	4	8
2	chr1	8	10	chr1	4	8
3	chr1	8	10	chr1	10	11

In [11]: `pb.visualize_intervals(overlapping_intervals)`





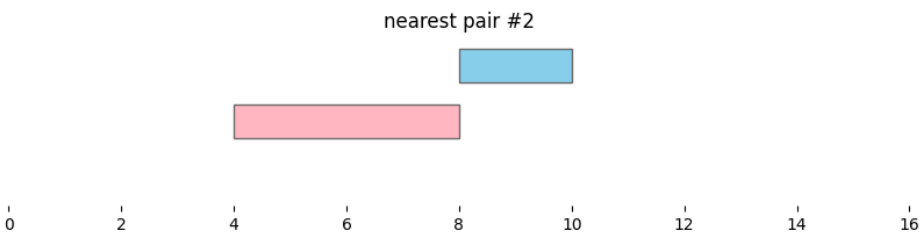
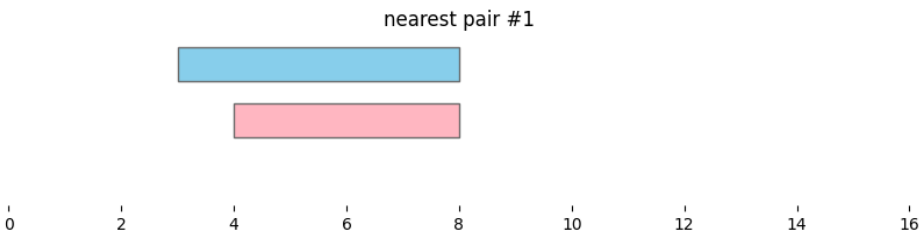
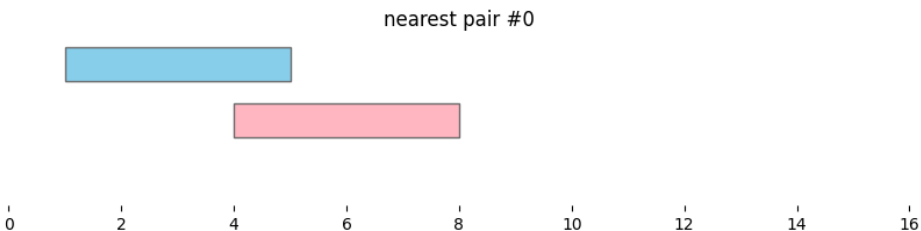
Nearest

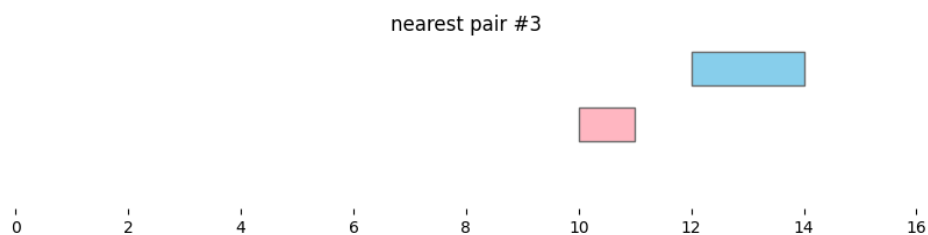
```
In [12]: nearest_intervals = pb.nearest(df1, df2,
output_type="pandas.DataFrame")
```

```
In [13]: display(nearest_intervals)
```

	chrom_1	start_1	end_1	chrom_2	start_2	end_2	distance
0	chr1	1	5	chr1	4	8	0
1	chr1	3	8	chr1	4	8	0
2	chr1	8	10	chr1	4	8	0
3	chr1	12	14	chr1	10	11	1

```
In [14]: pb.visualize_intervals(nearest_intervals, "nearest
pair")
```





Count overlaps

```
In [15]: count_overlaps = pb.count_overlaps(df1, df2,
      output_type="pandas.DataFrame")
```

```
In [16]: display(count_overlaps)
```

	chrom	start	end	count
0	chr1	1	5	1
1	chr1	3	8	1
2	chr1	8	10	2
3	chr1	12	14	0

Coverage

```
In [17]: coverage = pb.coverage(df1, df2,
      output_type="pandas.DataFrame")
```

```
In [18]: display(coverage)
```

	chrom	start	end	coverage
0	chr1	1	5	2
1	chr1	3	8	4
2	chr1	8	10	2
3	chr1	12	14	0

Merge

```
In [19]: merge = pb.merge(df1, output_type="pandas.DataFrame")
```

```
In [20]: display(merge)
```

	chrom	start	end	n_intervals
0	chr1	1	10	3
1	chr1	12	14	1

Reading bioinformatics data

GFF

BASIC READING WITH ATTRIBUTES UNNESTING

```
In [21]: gff = pb.scan_gff("data/example.gff3.bgz")
```

```
In [22]: gff.limit(3).collect()
```

0 rows [00:00, ?rows/s]

Out[22]: shape: (3, 9)

chrom	start	end	type	source	score	strand	phase
str	u32	u32	str	str	f32	str	u32
"chr1"	11869	14409	"gene"	"HAVANA"	null	"+"	null
"chr1"	11869	14409	"transcript"	"HAVANA"	null	"+"	null
"chr1"	11869	12227	"exon"	"HAVANA"	null	"+"	null

UNNESTING ATTRIBUTES

```
In [23]: gff =
pb.scan_gff("data/example.gff3.bgz").select(["start",
"end", "ID", "havana_transcript"])
```

```
In [24]: gff.limit(3).collect()
```

0 rows [00:00, ?rows/s]

Out[24]: shape: (3, 4)

start	end	ID	havana_transcript
u32	u32	str	str
11869	14409	"ENSG00000223972.5"	null
11869	14409	"ENST00000456328.2"	"OTTHUMT00000362751.1"
11869	12227	"exon:ENST00000456328.2:1"	"OTTHUMT00000362751.1"

VCF

In [25]: `pb.scan_vcf("data/example.vcf").limit(3).collect()` 

0 rows [00:00, ?rows/s]

Out [25]: shape: (2, 9)

chrom	start	end	id	ref	alt	qual	filter
str	u32	u32	str	str	str	f64	str
"chr21"	26960070	26960070	"rs116645811"	"G"	"A"	null	""
"chr21"	26965148	26965148	"rs1135638"	"G"	"A"	null	"" ["A synony

GET INFO FIELDS AVAILABLE IN THE VCF FILE

In [26]: `pb.describe_vcf("data/example.vcf").sort("name")` 

Out [26]: shape: (1, 3)

name	type	description
str	str	str
"CSQ"	"String"	"Consequence annotations from E..."

In [27]: `pb.scan_vcf("data/example.vcf", info_fields=["CSQ"]).limit(3).collect()` 

0 rows [00:00, ?rows/s]

Out [27]: shape: (2, 9)

chrom	start	end	id	ref	alt	qual	filter
str	u32	u32	str	str	str	f64	str
"chr21"	26960070	26960070	"rs116645811"	"G"	"A"	null	""
"chr21"	26965148	26965148	"rs1135638"	"G"	"A"	null	"" ["A synony

FASTQ

```
In [28]: pb.scan_fastq("data/example.fastq.gz").limit(3).collect()
```

0 rows [00:00, ?rows/s]

Out[28]: shape: (3, 4)

name		description	
str		str	
"SRR9130495.1"	"D00236:723:HG32CBCX2:1:1108:13..."	"NCAATACAAAAGCAATA1"	
"SRR9130495.2"	"D00236:723:HG32CBCX2:1:1108:14..."	"NGTCAAAGATAAGATCA1"	
"SRR9130495.3"	"D00236:723:HG32CBCX2:1:1108:17..."	"GTTTTCTCTGGTTATT1"	

BAM

```
In [29]: pb.scan_bam("data/example.bam").limit(3).collect()
```

0 rows [00:00, ?rows/s]

Out[29]: shape: (3, 11)

name	chrom	start	end	flags	cigar	mapp
str	str	u32	u32	u32	str	
"20FUKAAXX100202:1:21:2075:1360..."	"chr1"	1	101	1187	"101M"	
"20FUKAAXX100202:1:21:2733:1836..."	"chr1"	1	101	1123	"101M"	
"20FUKAAXX100202:1:22:19822:802..."	"chr1"	1	101	163	"101M"	

BED

```
In [30]: pb.scan_bed("data/example.bed.bgz").limit(3).collect()
```

0 rows [00:00, ?rows/s]

Out [30]: shape: (3, 4)

chrom	start	end	name
str	u32	u32	str
"chr16"	14800001	16800000	"FRA16A"
"chr16"	66700001	70800000	"FRA16B"
"chr16"	63917503	63934965	"FRA16C"

SQL processing

Registering tables

In [31]: `pb.register_gff("data/example.gff3.bgz", "example_gff")`

In [32]: `pb.sql("SHOW TABLES").collect()`

0 rows [00:00, ?rows/s]

Out [32]: shape: (18, 4)

table_catalog	table_schema	table_name	table_type
str	str	str	str
"datafusion"	"public"	"count_overlaps_coverage"	"LOCAL TEMPO"
"datafusion"	"public"	"example_gz"	"BASE T"
"datafusion"	"public"	"vcf_schema_975224660"	"BASE T"
"datafusion"	"public"	"s2"	"BASE T"
"datafusion"	"public"	"example_gff"	"BASE T"
...	...	...	...
"datafusion"	"information_schema"	"columns"	""
"datafusion"	"information_schema"	"df_settings"	""
"datafusion"	"information_schema"	"schemata"	""
"datafusion"	"information_schema"	"routines"	""
"datafusion"	"information_schema"	"parameters"	""

QUERYING REGISTERED TABLES

```
In [33]: pb.sql("DESCRIBE example_gff").collect()
```



0 rows [00:00, ?rows/s]

Out[33]: shape: (9, 3)

column_name	data_type	is_nullable
str	str	str
"chrom"	"Utf8"	"NO"
"start"	"UInt32"	"NO"
"end"	"UInt32"	"NO"
"type"	"Utf8"	"NO"
"source"	"Utf8"	"NO"
"score"	"Float32"	"YES"
"strand"	"Utf8"	"NO"
"phase"	"UInt32"	"YES"
"attributes"	"List(Field { name: 'item', dat..."	"YES"

```
In [34]: pb.sql("SELECT start, end, type FROM example_gff WHERE end = 14409").collect()
```



0 rows [00:00, ?rows/s]

Out[34]: shape: (2, 3)

start	end	type
u32	u32	str
11869	14409	"gene"
11869	14409	"transcript"

REGISTERING VIEWS

```
In [35]: pb.register_view("v_example_gff", "SELECT start, end, type FROM example_gff WHERE end = 14409")
```



```
In [36]: pb.sql("SELECT * FROM v_example_gff").collect()
```



0 rows [00:00, ?rows/s]

Out[36]: shape: (2, 3)

start	end	type
u32	u32	str
11869	14409	"gene"

start	end	type
u32	u32	str
11869	14409	"transcript"

Working with cloud storage

In [38]: `pb.describe_vcf("gs://gcp-public-data--gnomad/release/4.1/genome_sv/gnomad.v4.1.sv.sites.vcf.gz").filter(pl.col("description").str.contains(r"Latino.* allele frequency"))` 

Out[38]: shape: (3, 3)

name	type	description
str	str	str
"AF_amr"	"Float"	"Latino allele frequency (biall..."
"AF_amr_XY"	"Float"	"Latino XY allele frequency (bi..."
"AF_amr_XX"	"Float"	"Latino XX allele frequency (bi..."

In [39]: `pb.register_vcf("gs://gcp-public-data--gnomad/release/4.1/genome_sv/gnomad.v4.1.sv.sites.vcf.gz", "gnomad", info_fields=['AF_amr'])` 
`query = """`
 `SELECT`
 `chrom,`
 `start,`
 `end,`
 `alt,`
 `array_element(af_amr,1) AS af_amr`
 `FROM gnomad`
 `WHERE`
 `filter = 'HIGH_NCR'`
 `AND`
 `alt = '<DUP>'`
 `"""`
`pb.sql(f"{query} LIMIT 3").collect()`

0rows [00:00, ?rows/s]

Out[39]: shape: (3, 5)

chrom	start	end	alt	af_amr
str	u32	u32	str	f32
"chr1"	10000	295666	"<DUP>"	0.000293
"chr1"	138000	144000	"<DUP>"	0.000166

chrom	start	end	alt	af_amr
str	u32	u32	str	f32
"chr1"	160500	172100	"<DUP>"	0.002639

OVERLAPPING WITH A LOCAL DATAFRAME

In [40]: `pb.register_vcf("s3://gnomad-public-us-east-1/release/4.1/vcf/exomes/gnomad.exomes.v4.1.sites.chr1.vcf.bgz", "gnomad_sites_chr1", info_fields=[])` 

In [41]: `pb.sql("SELECT chrom, start, end, alt FROM gnomad_sites_chr1 LIMIT 10").collect()` 

0rows [00:00, ?rows/s]

Out[41]: shape: (10, 4)

chrom	start	end	alt
str	u32	u32	str
"chr1"	11994	11994	"C"
"chr1"	12016	12016	"A"
"chr1"	12060	12065	"C"
"chr1"	12074	12074	"C"
"chr1"	12102	12102	"A"
"chr1"	12106	12106	"G"
"chr1"	12138	12138	"A"
"chr1"	12158	12158	"T"
"chr1"	12165	12165	"A"
"chr1"	12168	12168	"G"

In [42]: `pb.register_view("v_gnomad_sites_chr1", "SELECT * FROM gnomad_sites_chr1 LIMIT 20")` 

In [43]: `pb.sql("SELECT * FROM v_gnomad_sites_chr1").collect()` 

0rows [00:00, ?rows/s]

Out[43]: shape: (20, 8)

chrom	start	end	id	ref	alt	qual	filter
str	u32	u32	str	str	str	f64	str
"chr1"	11994	11994	"	"T"	"C"	null	"AC0;AS_VQSR"

chrom	start	end	id	ref	alt	qual	filter
str	u32	u32	str	str	str	f64	str
"chr1"	12016	12016	""	"G"	"A"	null	"AC0;AS_VQSR"
"chr1"	12060	12065	""	"CTGGAG"	"C"	null	"AC0;AS_VQSR"
"chr1"	12074	12074	""	"T"	"C"	null	"AC0;AS_VQSR"
"chr1"	12102	12102	""	"G"	"A"	null	"AC0;AS_VQSR"
...	...	...	...	...	...	...	...
"chr1"	12198	12198	"rs62635282"	"G"	"C"	null	"AS_VQSR"
"chr1"	12201	12201	""	"C"	"G"	null	"AS_VQSR"
"chr1"	12214	12214	"rs202068986"	"C"	"G"	null	"AC0"
"chr1"	12225	12225	""	"C"	"T"	null	"AS_VQSR"
"chr1"	12235	12235	""	"G"	"A"	null	"AS_VQSR"

```
In [44]: df = pl.DataFrame({
    "chrom": ["chr1", "chr1"],
    "start": [11993, 12102],
    "end": [11996, 12200],
    "annotation": ["ann1", "ann2"]
})
pb.from_polars("test_annotation", df)
pb.sql("SELECT * FROM test_annotation").collect()
```

0rows [00:00, ?rows/s]

Out[44]: shape: (2, 4)

chrom	start	end	annotation
str	i64	i64	str
"chr1"	11993	11996	"ann1"
"chr1"	12102	12200	"ann2"

```
In [45]: pb.overlap("v_gnomad_sites_chr1", "test_annotation").limit(
    3).collect()
```

0rows [00:00, ?rows/s]

Out[45]: shape: (3, 12)

chrom_1	start_1	end_1	chrom_2	start_2	end_2	annotation_1	id_2	ref_2
str	u32	u32	str	i64	i64	str	str	str
"chr1"	11994	11994	"chr1"	11993	11996	"ann1"	""	"T"
"chr1"	12102	12102	"chr1"	12102	12200	"ann2"	""	"G"
"chr1"	12106	12106	"chr1"	12102	12200	"ann2"	""	"T"

Parallel processing

These demo datasets are from [databio.zip](#) benchmark.

```
In [46]: pb.set_option("datafusion.execution.target_partitions"
"1")
```

```
In [47]: %%time
pb.overlap("data/ex-rna/*.parquet",
"data/chainRn4/*.parquet", cols1=cols,
cols2=cols).collect().count()
```

0rows [00:00, ?rows/s]

CPU times: user 3.13 s, sys: 566 ms, total: 3.69 s

Wall time: 3.72 s

Out[47]: shape: (1, 6)

contig_1	pos_start_1	pos_end_1	contig_2	pos_start_2	pos_end_2
u32	u32	u32	u32	u32	u32
164214743	164214743	164214743	164214743	164214743	164214743

```
In [48]: pb.set_option("datafusion.execution.target_partitions"
"2")
```

```
In [50]: %%time
pb.overlap("data/ex-rna/*.parquet",
"data/chainRn4/*.parquet", cols1=cols,
cols2=cols).collect().count()
```

0rows [00:00, ?rows/s]

CPU times: user 3.2 s, sys: 556 ms, total: 3.76 s

Wall time: 1.95 s

Out[50]: shape: (1, 6)

contig_1	pos_start_1	pos_end_1	contig_2	pos_start_2	pos_end_2
u32	u32	u32	u32	u32	u32
164214743	164214743	164214743	164214743	164214743	164214743

Parallel reading of BGZF compressed FASTQ files

```
In [51]: %%bash
gsutil cat gs://genomics-public-data/platinum-
```

```
genomes/fastq/ERR194146.fastq.gz | gunzip -c | bgzip -c > /tmp/ERR194146.fastq.bgz
cd /tmp && bgzip -r /tmp/ERR194146.fastq.bgz
```

```
/opt/homebrew/share/google-cloud-sdk/platform/gsutil/third-party/google-auth-library-python/google/__init__.py:18: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools <81.
import pkg_resources
```

```
In [ ]: pb.set_option("datafusion.execution.target_partitions", "1")
```

```
In [59]: %%time
pb.scan_fastq("/tmp/ERR194146.fastq.bgz",
parallel=True).count().collect()
```

```
0rows [00:00, ?rows/s]
```

```
CPU times: user 3.71 s, sys: 242 ms, total: 3.95 s
```

```
Wall time: 3.77 s
```

```
Out[59]: shape: (1, 4)
```

name	description	sequence	quality_scores
u32	u32	u32	u32
8657652	8657652	8657652	8657652

```
In [53]: pb.set_option("datafusion.execution.target_partitions", "2")
```

```
In [55]: %%time
pb.scan_fastq("/tmp/ERR194146.fastq.bgz",
parallel=True).count().collect()
```

```
0rows [00:00, ?rows/s]
```

```
CPU times: user 3.83 s, sys: 344 ms, total: 4.17 s
```

```
Wall time: 2.03 s
```

```
Out[55]: shape: (1, 4)
```

name	description	sequence	quality_scores
u32	u32	u32	u32
8657652	8657652	8657652	8657652

```
In [60]: pb.set_option("datafusion.execution.target_partitions", "1")
```

```
"4")
```

```
In [61]: %%time
pb.scan_fastq("/tmp/ERR194146.fastq.bgz",
parallel=True).count().collect()

0rows [00:00, ?rows/s]
CPU times: user 3.86 s, sys: 397 ms, total: 4.26 s
Wall time: 1.23 s

Out[61]: shape: (1, 4)
```

name	description	sequence	quality_scores
u32	u32	u32	u32
8657652	8657652	8657652	8657652

```
In [62]: pb.set_option("datafusion.execution.target_partitions"
"1")
```

Streaming

Make sure you restart the kernel before running the next cells. [memory-profiler](#) is required.

```
In [3]: %load_ext memory_profiler
```

OVERLAP IN THE STREAMING MODE

```
In [4]: %memit pb.overlap("data/ex-rna/*.parquet",
"data/chainRn4/*.parquet", cols1=cols,
cols2=cols).sink_parquet("/tmp/overlap.parquet")
```

```
0rows [00:00, ?rows/s]
peak memory: 1399.36 MiB, increment: 1186.92 MiB
```

OVERLAP WITH MATERIALIZATION

```
In [65]: %memit pb.overlap("data/ex-rna/*.parquet",
"data/chainRn4/*.parquet", cols1=cols,
cols2=cols).collect().write_parquet("/tmp/overlap.parquet")
```

```
0rows [00:00, ?rows/s]
peak memory: 17261.36 MiB, increment: 4621.69 MiB
```

Working with zero-based coordinates

Compatibility with zero-based coordinates software, for example **bioframe**.

```
In [5]: BIO_PD_DF1 = pd.read_parquet(f"data/exons/")
        BIO_PD_DF2 = pd.read_parquet(f"data/fBrain-DS14718/")
```

```
In [6]: bf_overlap = bf.overlap(
        BIO_PD_DF1,
        BIO_PD_DF2,
        cols1=cols,
        cols2=cols,
        suffixes=("_1", "_2"),
        how="inner",
    )
```

```
In [7]: pb_overlap = pb.overlap(
        BIO_PD_DF1,
        BIO_PD_DF2,
        cols1=cols,
        cols2=cols,
        output_type="pandas.DataFrame",
        suffixes=("_1", "_2"),
    )
```

Since polars-bio is one-based, the assertion will **fail**.

```
In [8]: assert len(bf_overlap) == len(pb_overlap)
```

```
-----
-----
AssertionError                                Traceback (most recent call last)
Cell In[8], line 1
----> 1 assert len(bf_overlap) == len(pb_overlap)

AssertionError:
```

To use polars-bio with zero-based coordinates, set `use_zero_based=True`:

```
In [9]: pb_overlap = pb.overlap(
        BIO_PD_DF1,
        BIO_PD_DF2,
```

```

cols1=cols,
cols2=cols,
output_type="pandas.DataFrame",
suffixes=("_1", "_2"),
use_zero_based=True,
)

```

WARNING:polars_bio:0-based coordinate system was selected. Please ensure that both datasets follow this coordinate system.

In [10]: `assert len(bf_overlap) == len(pb_overlap)`

Example - gnomAD VCF file reading

1. How to read gnomAD VCF files from Google Cloud Storage or AWS S3

Google Cloud Storage

In [11]: `import polars_bio as pb`
`import polars as pl`

In [12]: `gcs_vcf_path = (`
 `"gs://gcp-public-data--`
`gnomad/release/4.1/genome_sv/gnomad.v4.1.sv.sites.vcf.g`
`z"`
`)`

In [15]: `# we need to override the compression type as the file is`
`bgzipped not gzipped as the extension suggests`
`pb.scan_vcf(gcs_vcf_path, info_fields=`
 `[]).limit(3).collect()`

0rows [00:00, ?rows/s]

Out [15]: shape: (3, 8)

chrom	start	end	id	ref	alt	
str	u32	u32	str	str	str	
"chr1"	10000	295666	"gnomAD-SV_v3_DUP_chr1_01c2781c"	"N"	"<DUP>"	1
"chr1"	10434	10434	"gnomAD-SV_v3_BND_chr1_1a45f73a"	"N"	"<BND>"	2
"chr1"	10440	10440	"gnomAD-SV_v3_BND_chr1_3fa36917"	"N"	"<BND>"	1

In [16]: `pb.describe_vcf(gcs_vcf_path).sort("name").limit(5)` 

Out [16]: shape: (5, 3)

name	type	description
str	str	str
"AC"	"Integer"	"Number of non-reference allele..."
"AC_XX"	"Integer"	"Number of non-reference XX all..."
"AC_XY"	"Integer"	"Number of non-reference XY all..."
"AC_afr"	"Integer"	"Number of non-reference Africa..."
"AC_afr_XX"	"Integer"	"Number of non-reference Africa..."

AWS S3

In [17]: `# here we can use automatic compression detection`
`aws_s3_vcf_path = "s3://gnomad-public-us-east-1/release/4.1/vcf/exomes/gnomad.exomes.v4.1.sites.chr21.vcf.bgz"` 

In [18]: `pb.scan_vcf(aws_s3_vcf_path, chunk_size=8,`
`concurrent_fetches=1, info_fields=`
 `[]).limit(3).collect()` 

0rows [00:00, ?rows/s]

Out [18]: shape: (3, 8)

chrom	start	end	id	ref	alt	qual	filter
str	u32	u32	str	str	str	f64	str
"chr21"	5031905	5031905	""	"C"	"A"	null	"AC0;AS_VQSR"
"chr21"	5031905	5031905	""	"C"	"T"	null	"AC0;AS_VQSR"
"chr21"	5031909	5031909	""	"T"	"C"	null	"AC0;AS_VQSR"

2. How to specify additional VCF INFO fields to be parsed

```
In [19]: vcf_info_fields = ["SVTYPE", "SVLEN"]
         pb.scan_vcf(gcs_vcf_path,
         info_fields=vcf_info_fields).limit(3).collect()
```

0 rows [00:00, ?rows/s]

Out[19]: shape: (3, 10)

chrom	start	end		id	ref	alt	
str	u32	u32		str	str	str	
"chr1"	10000	295666	"gnomAD-SV_v3_DUP_chr1_01c2781c"	"N"	"<DUP>"		1
"chr1"	10434	10434	"gnomAD-SV_v3_BND_chr1_1a45f73a"	"N"	"<BND>"		2
"chr1"	10440	10440	"gnomAD-SV_v3_BND_chr1_3fa36917"	"N"	"<BND>"		1

3. How to speed up reading local compressed VCF files with multiple threads

```
In [20]: ! gsutil -m cp $gcs_vcf_path
         /tmp/gnomad.v4.1.sv.sites.vcf.gz &> /dev/null
```

```
In [22]: %%time
         pb.scan_vcf("/tmp/gnomad.v4.1.sv.sites.vcf.gz",
         thread_num=1, info_fields=[]).count().collect()
```

0 rows [00:00, ?rows/s]

CPU times: user 11.8 s, sys: 2.07 s, total: 13.8 s

Wall time: 9.99 s

Out[22]: shape: (1, 8)

chrom	start	end	id	ref	alt	qual	filt
u32	u32	u32	u32	u32	u32	u32	u32
2154486	2154486	2154486	2154486	2154486	2154486	2153697	2154486

```
In [23]: %%time
         pb.scan_vcf("/tmp/gnomad.v4.1.sv.sites.vcf.gz",
         thread_num=4, info_fields=[]).count().collect()
```

0 rows [00:00, ?rows/s]

CPU times: user 11.4 s, sys: 1.63 s, total: 13 s
 Wall time: 3.09 s

Out[23]: shape: (1, 8)

chrom	start	end	id	ref	alt	qual	filt
u32	u32	u32	u32	u32	u32	u32	u32
2154486	2154486	2154486	2154486	2154486	2154486	2153697	2154486

4. How to perform an overlap operation on two remote VCF files in streaming mode

```
In [24]: vcf_1 = "gs://gcp-public-data--
          gnomad/release/4.1/genome_sv/gnomad.v4.1.sv.sites.vcf.gz"
          vcf_2 = "gs://gcp-public-data--
          gnomad/release/4.1/vcf/exomes/gnomad.exomes.v4.1.sites.
          chr21.vcf.bgz"
```

```
In [25]: object_storage_options = pb.ObjectStorageOptions(
          allow_anonymous=True,
          enable_request_payer=False,
          chunk_size=64,
          concurrent_fetches=8,
          max_retries=5,
          timeout=10,
          compression_type="bgz",
        )
          vcf_read_options_1 = pb.VcfReadOptions(
            info_fields=["SVTYPE", "SVLEN"],
            thread_num=1,
            object_storage_options=object_storage_options,
          )
          vcf_read_options_2 = pb.VcfReadOptions(
            object_storage_options=object_storage_options,
          )
          read_options_1 =
          pb.ReadOptions(vcf_read_options=vcf_read_options_1)
          read_options_2 =
          pb.ReadOptions(vcf_read_options=vcf_read_options_2)
```

```
In [26]: pb.overlap(vcf_1, vcf_2, read_options1=read_options_1,
                    read_options2=read_options_2).sink_csv(
                    "/tmp/streaming_run.csv"
                    )
```

0rows [00:00, ?rows/s]

In [27]: `pl.read_csv("/tmp/streaming_run.csv").limit(3)`

Out[27]: shape: (3, 18)

chrom_1	start_1	end_1	id_1	ref_1	alt_1	qual_1	filter_1	sv
str	i64	i64	str	i64	i64	str	str	
"chr21"	5019150	5047500	"chr21"	5036183	5036183	""	"A"	
"chr21"	5019150	5047500	"chr21"	5036184	5036184	""	"G"	
"chr21"	5019150	5047500	"chr21"	5036185	5036185	""	"G"	

In [28]: `pb.overlap(vcf_1, vcf_2, read_options1=read_options_1, read_options2=read_options_2).collect().count()`

0rows [00:00, ?rows/s]

Out[28]: shape: (1, 18)

chrom_1	start_1	end_1	chrom_2	start_2	end_2	id_1
u32	u32	u32	u32	u32	u32	u32
17882545	17882545	17882545	17882545	17882545	17882545	17882545

5. How to read a VCF from Google Life Sciences

In [29]: `gcs_vcf_path = "gs://genomics-public-data/platinum-genomes/vcf/NA12878_S1.genome.vcf"`

In [30]: `info_fields=["AC", "AF"]`

In [31]: `pb.scan_vcf(gcs_vcf_path, info_fields=info_fields).limit(3).collect()`

0rows [00:00, ?rows/s]

Out[31]: shape: (3, 10)

chrom	start	end	id	ref	alt	qual	filter
str	u32	u32	str	str	str	f64	str
"chrM"	1	1	""	"G"	""	null	"PASS"
"chrM"	2	72	""	"A"	""	null	"PASS"

chrom	start	end	id	ref	alt	qual	filter
str	u32	u32	str	str	str	f64	s1
"chrM"	73	73	"	"G"	"A"	8752.780273	"TruthSensitivityTranche99.90to.

6. SQL data processing

Check [SQL reference](#) for details.

```
In [32]: pb.register_vcf(vcf_1, "gnomad_sv", thread_num=1,
info_fields=["SVTYPE", "SVLEN"])
```

```
In [33]: pb.sql("SELECT chrom, svtype FROM
gnomad_sv").limit(3).collect()
```

0rows [00:00, ?rows/s]

Out[33]: shape: (3, 2)

chrom	svtype
str	str
"chr1"	"DUP"
"chr1"	"BND"
"chr1"	"BND"

```
In [34]: pb.sql("SELECT * FROM gnomad_sv WHERE SVTYPE = 'DEL'
AND SVLEN > 1000").limit(3).collect()
```

0rows [00:00, ?rows/s]

Out[34]: shape: (3, 10)

chrom	start	end	id	ref	alt
str	u32	u32	str	str	str
"chr1"	22000	30000	"gnomAD-SV_v3_DEL_chr1_fa103016"	"N"	"<D
"chr1"	40000	47000	"gnomAD-SV_v3_DEL_chr1_b26f63f7"	"N"	"<D
"chr1"	79086	88118	"gnomAD-SV_v3_DEL_chr1_733c4ef0"	"N"	"<DEL:ME:LIN

```
In [35]: pb.sql("SELECT alt, count(*) as cnt FROM gnomad_sv
group by alt").collect_schema()
```

Out[35]: Schema([('alt', String), ('cnt', Int64)])

In [36]: `pb.sql("SELECT alt, count(*) as cnt FROM gnomad_sv
group by alt").collect()`

0rows [00:00, ?rows/s]

Out[36]: shape: (13, 2)

alt	cnt
str	i64
"<DUP>"	269326
"<BND>"	356035
"<CNV>"	721
""	1197080
"<INS>"	83441
...	...
"<INS:ME:SVA>"	17607
"<CPX>"	15189
"<INV>"	2193
"<DEL:ME:HERVK>"	693
"<CTX>"	99

In [37]: `pb.sql("SELECT chrom, count(*) as cnt FROM gnomad_sv
GROUP BY chrom ORDER BY chrom").collect()`

0rows [00:00, ?rows/s]

Out[37]: shape: (24, 2)

chrom	cnt
str	i64
"chr1"	182804
"chr10"	96755
"chr11"	95690
"chr12"	97655
"chr13"	63839
...	...
"chr7"	131866
"chr8"	101224
"chr9"	87748
"chrX"	78076
"chrY"	12488

```
In [38]: pb.sql("SELECT * FROM gnomad_sv WHERE
             chrom='chr1'").sink_csv("/tmp/gnomad_chr1.csv")
```

0rows [00:00, ?rows/s]

```
In [39]: pl.read_csv("/tmp/gnomad_chr1.csv").count()
```

Out[39]: shape: (1, 10)

chrom	start	end	id	ref	alt	qual	filter	svtype
u32	u32	u32	u32	u32	u32	u32	u32	u32
182804	182804	182804	182804	182804	182804	182765	182804	182804

```
In [40]: pb.register_vcf(vcf_2, "gnomad_exomes", info_fields=
             ["AC", "AF"])
```

```
In [41]: pb.sql("SELECT replace(chrom,'chr','') AS chrom, start,
             ac,af FROM gnomad_exomes WHERE
             array_element(af,1)>0.01").limit(10).collect()
```

0rows [00:00, ?rows/s]

Out[41]: shape: (10, 4)

chrom	start	ac	af
str	u32	list[i32]	list[f32]
"21"	5033364	[372992]	[0.337086]
"21"	5033539	[1107064]	[0.996312]
"21"	5034629	[15145]	[0.020082]
"21"	5035021	[1811]	[0.307888]
"21"	5035108	[2]	[0.010989]
"21"	5035658	[255555]	[0.336447]
"21"	5035846	[37233]	[0.286906]
"21"	5035921	[1682]	[0.010757]
"21"	5116593	[4032]	[0.018626]
"21"	5116760	[101]	[0.014914]

```
In [42]: pb.overlap("gnomad_sv",
             "gnomad_exomes").collect().count()
```

0rows [00:00, ?rows/s]

Out[42]: shape: (1, 20)

chrom_1	start_1	end_1	chrom_2	start_2	end_2	id_1
u32	u32	u32	u32	u32	u32	u32
17882545	17882545	17882545	17882545	17882545	17882545	17882545